

Manual Técnico Staticdata

- Manual Técnico — Staticdata
 - 1. Stack y versiones
 - 2. Arquitectura
 - 3. Inyección de dependencias
 - 4. Base de datos (Room)
 - 5. Motor de dominio: StatsEngine
 - 6. Evaluación de ejercicios
 - 7. Progresión y gamificación
 - 8. UI / Jetpack Compose
 - 9. Permisos y manifiesto
 - 10. Build
 - 11. Pruebas
 - 12. Mantenimiento y ampliaciones

Manual Técnico — Staticdata

1. Stack y versiones

Componente	Versión
Kotlin	2.0.20
Android Gradle Plugin	8.5.2
Gradle	8.7
KSP	2.0.20-1.0.25
Compose BOM	2024.09.00
Material3	1.2.1
Navigation Compose	2.8.0
Room	2.6.1
Coroutines	1.8.1
JDK	17
compileSdk / targetSdk	34
minSdk	24

Todas las versiones están fijadas explícitamente (sin + ni latest) en build.gradle.kts / app/build.gradle.kts.

2. Arquitectura

MVVM + Repository Pattern en tres capas:

data/	
local/entity/	17 entidades Room
local/dao/	9 interfaces DAO
local/converters/	StringListConverter (List<String> <-> String)
local/seed/	SeedSpecs.kt + SeedProvider.kt (contenido inicial)
local/	StaticdataDatabase.kt (Room)
repository/	6 repositorios (mapean entidad <-> modelo de dominio)
domain/	
model/	DomainModels.kt (clases puras, sin Android)
logic/	StatsEngine, ExerciseEvaluator, ProgressRules
usecase/	orquestación multi-repositorio (UseCases.kt)

ui/	
navigation/	NavGraph + Routes
theme/	Color/Type/Theme
illustration/	Ilustraciones Canvas (mascota, insignias, avatares, iconos)
components/	Componentes compartidos (barra XP, chips, gráfico de barras...)
screens/<módulo>/	ViewModel + Screen por cada uno de los 11 módulos

Regla de dependencia: ui depende de domain y data; data puede depender de domain (usa StatsEngine al sembrar contenido); domain no depende de Android ni de Room — por eso es 100% testeable con JUnit plano.

3. Inyección de dependencias

Sin frameworks externos (Hilt/Koin) para mantener el proyecto simple: AppContainer (en util/AppContainer.kt) crea perezosamente la base de datos, los repositorios y los casos de uso; se expone a los Composables mediante CompositionLocalProvider(LocalAppContainer provides container) desde MainActivity. Cada pantalla obtiene su ViewModel con rememberAppViewModel { ... }, una fábrica mínima que evita boilerplate.

4. Base de datos (Room)

Ver docs/BASE_DE_DATOS.md para el modelo completo. Puntos clave:

- StaticdataDatabase (versión 1, exportSchema = true) registra las 17 entidades y expone 9 DAOs.
- Los campos List<String> (opciones de ejercicio, respuesta correcta, etiquetas de una muestra) se serializan con StringListConverter, que usa el carácter de control U+001F como separador para no chocar con comas o texto libre en español.
- SeedProvider.seedIfEmpty() puebla la base la primera vez que arranca la app (disparado desde StaticdataApp.onCreate, en una corrutina de Dispatchers.IO), generando 30 datasets, 10 casos con 30 ejercicios (cuyas respuestas correctas se calculan con StatsEngine sobre los datos generados, no se escriben a mano), 3 experimentos de muestreo y 10 insignias.

5. Motor de dominio: StatsEngine

domain/logic/StatsEngine.kt es un object sin dependencias de Android:

- count, frequencyTable, frequencyOf, percentage, mode, mean, range: cálculos estadísticos deterministas.
- sample(population, size, seed): muestreo aleatorio sin reemplazo, reproducible dado el mismo seed (usa kotlin.random.Random(seed)).
- sampleVariability(runs): compara la proporción de cada etiqueta entre varias tiradas y calcula el rango de variación observado.
- validateSurveyQuestion / validateSurveyOptions: reglas de negocio (pregunta ≤ 80 caracteres; 2-6 opciones sin duplicados).

Todas las funciones lanzan StatsEngine.InvalidDataException ante datos inválidos (población vacía, tamaño de muestra mayor que la población, total ≤ 0 al calcular un porcentaje, etc.), en vez de fallar en silencio.

6. Evaluación de ejercicios

domain/logic/ExerciseEvaluator.kt compara la respuesta dada contra Exercise.correctAnswer según el tipo: comparación de secuencia exacta para ORDENAR_FRECUENCIA/CLASIFICAR_TIPO, comparación de conjunto para el resto. Nunca compara texto crudo sin normalizar (recorta espacios y compara en minúsculas).

7. Progresión y gamificación

domain/logic/ProgressRules.kt define XP por ejercicio correcto (+10, o +15 si fue a la primera), XP por caso completado (+30), XP por tirada de muestreo (+4), umbrales de nivel (8 niveles) y la evaluación de desbloqueo de insignias comparando el progreso actual contra el requirement de

cada Badge (formato "TIPO:VALOR", p. ej. CASES:3).

`domain/usecase/UseCases.kt` (`SubmitExerciseAnswerUseCase`) orquesta el flujo completo de responder un ejercicio: evalúa, registra el intento, actualiza XP/nivel, decide si el caso pasa a COMPLETADO o DOMINADO (según si todos los ejercicios se acertaron también a la primera), refresca qué casos bloqueados ya alcanzan el nivel mínimo, y comprueba desbloqueo de insignias.

8. UI / Jetpack Compose

- Tema propio (`ui/theme`) con paleta “agencia de detectives” (tinta nocturna, pergamino, ámbar, turquesa) y tipografía con pesos marcados.
- Ilustraciones 100% vectoriales dibujadas con `Canvas` de Compose (`ui/illustration`): mascota Dati (4 estados de ánimo), 10 insignias, 8 avatares, 11 iconos de módulo, fondo decorativo de “constelación de datos” — cero dependencia de imágenes remotas.
- `ui/components/FrequencyBarChart.kt`: gráfico de barras animado que dibuja directamente una `FrequencyTable` real (nunca datos de relleno).
- Navegación: `NavGraph.kt` decide la pantalla inicial (Onboarding u Oficina de casos) según `UserProfile.onboardingCompleted`.

9. Permisos y manifiesto

`AndroidManifest.xml` no declara ningún permiso: no hay `INTERNET`, `CAMERA`, `RECORD_AUDIO`, `ACCESS_FINE_LOCATION` ni similares, porque ninguna función de la app los necesita.

10. Build

```
./gradlew clean
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
```

Ver `docs/BUILD_REPORT.md` para el estado real de estos comandos en el entorno de generación (sin Android SDK ni red) y `.github/workflows/android-build.yml` para obtener una compilación verificada en CI.

11. Pruebas

71 pruebas en `app/src/test`:

- `domain/StatsEngineTest.kt` (33): todas las funciones del motor, incluyendo casos límite (población vacía, tamaño de muestra excesivo, porcentaje con total 0, empates alfabéticos, valores con espacios).
- `domain/ExerciseEvaluatorTest.kt` (8): las 6 mecánicas de evaluación.
- `domain/ProgressRulesTest.kt` (12): niveles, XP, desbloqueo de insignias por distintos requisitos.
- `data/StringListConverterTest.kt` (4): serialización ida y vuelta.
- `data/DatabaseSeedTest.kt` (8, Robolectric + Room en memoria): cantidad exacta de `datasets/casos/ejercicios/insignias/experimentos` sembrados, idempotencia del seeding, ejercicios con respuesta no vacía.
- `data/SurveyRepositoryTest.kt` (6, Robolectric + Room en memoria): validaciones de encuesta, cálculo de tabla de frecuencias sobre respuestas reales, caché de resultados.

12. Mantenimiento y ampliaciones

- Añadir un dataset nuevo: agregar una entrada a `DatasetSeedCatalog` en `SeedSpecs.kt` (categórico o numérico).
- Añadir un caso nuevo: agregar un `CaseBlueprint` en `SeedProvider.caseBlueprints()` apuntando a un `specIndex` existente; los ejercicios se calculan automáticamente a partir de los datos reales de ese dataset.
- Añadir un tipo de ejercicio: extender `ExerciseType`, añadir su rama en `ExerciseEvaluator.evaluate` y en `ExerciseInteraction` (UI).

- Añadir una insignia: agregar una fila en `SeedProvider.seedBadges()` con su requirement (`CASES:n`, `EXERCISES:n`, `FIRST_TRY:n`, `LEVEL:n`, `SAMPLES:n` O `SURVEYS:n`) y su dibujo en `BadgeArt.kt`.